

Contents

Build-Phase Engineering Plan	1
Nestly — Childcare Industry Pack (v1)	1
0. How to use this document	1
1. Getting the validated frontend into a real repository	1
2. Architecture overview	2
3. Data model → schema	2
4. API surface (high-level)	3
5. Compliance rule pack — from spec to implementation	3
6. Phased engineering roadmap	4
7. Security, privacy, and audit requirements	4
8. Decision log (this round)	5
9. Open items	5

Build-Phase Engineering Plan

Nestly — Childcare Industry Pack (v1)

Prepared for Gene · August 31, 2026 Status: **Follow-up to the Unified PRD (Section 14)**. Design is validated (Figma Make prototype, punch list addressed); this document covers everything the PRD deliberately deferred — backend, data layer, integrations, hosting, and the engineering sequence to get from validated design to a working product.

0. How to use this document

Same audience model as the PRD: a future Claude session, a developer, or you six months from now should be able to open this and know exactly what’s built, what’s next, and what’s still undecided. Section 8 is the decision log; Section 9 is what’s genuinely open.

Starting position, confirmed this round: the Figma Make output (Vite + React + TypeScript + Tailwind, 9 screens) is the real frontend foundation — not a throwaway reference. The build phase extends it with a real backend and data layer rather than rewriting the frontend.

1. Getting the validated frontend into a real repository

This is the first practical blocker, and it’s an action item for you, not something I can complete from inside this connector. The Figma MCP connection available here can browse the Make file’s structure and, on /design/ files, pull design tokens — but for /make/ files it does not expose raw file contents I can pull directly into a repo (confirmed while reviewing the prototype earlier).

Action needed: use Figma Make’s own export path — either its GitHub sync/connect feature or a direct code export/download — to get the src/ tree (App.tsx, the 9 page components, data.ts, index.css, package.json, etc.) into an actual Git repository. Once that repository exists (locally or on GitHub), I have full file access through normal tools and can work in it directly — read every component, wire up real data, add the backend, and so on. Everything else in this document assumes that repository exists.

2. Architecture overview

Confirms and refines the recommendation from the earlier MVP build plan, now that a real frontend exists to build against:

Layer	Decision	Why
Frontend	Keep Figma Make's Vite + React + TypeScript + Tailwind output	Already built, already validated against the PRD and WCAG considerations — rebuilding it would throw away real, reviewed work
Backend	Node.js + TypeScript, REST + WebSockets	Shares types with the frontend (same language), WebSockets needed for live ratio updates and real-time alerts (Section 9.2 workflows 2 and 6)
Database	PostgreSQL	Relational integrity matters for billing, ratios, and compliance records; strong consistency guarantees
Offline sync	Local IndexedDB (via a sync library) on-device, conflict-resolution sync layer	Required by the PRD's offline-first non-functional requirement (Section 12) — must be designed in now, not retrofitted
Auth	Session-based or JWT, role-scoped to Tenant → Facility → Group	Must enforce the Section 4.2/12.2 role matrix at the API layer, not just hide UI elements client-side
Payments	Stripe, added in a later build sub-phase (Section 6)	Matches Playground's and Mindbody's own choice (both researched earlier); real integration is explicitly out of scope until the mocked billing flow is replaced
Hosting	Containerized, AWS or GCP	Standard, scalable, easy to staff

3. Data model → schema

Directly implements the entities from PRD Section 8. Each becomes a Postgres table; foreign keys enforce the relationships already specified.

- **tenants** (organization: Sunshine Childcare Group) — holds state/jurisdiction, billing plan, branding
- **facilities** (centers: Coral Springs, Boca Raton, Pompano Beach) — belongs to a tenant
- **groups** (rooms: Lily Infants, Violet Toddlers, Iris Preschool) — belongs to a facility, holds age-band and ratio-max
- **people** (children) — belongs to a facility via enrollment; child-specific fields (DOB → drives age-band/ratio bucket)

- guardians + person_guardians (join table) — relationship type, access permissions
 - staff — belongs to a tenant, assignable to facilities/groups; holds role
 - certifications — belongs to staff; type (background screening, CPR, etc.), issue/expiry dates — this is where the Section 10 background-screening 5-year rescreening logic lives
 - enrollments — links a person to a facility/group over time; status (waitlisted/active/withdrawn)
 - attendance_events — check-in/out timestamps, tied to person + facility + staff-on-duty
 - daily_records — flexible structured entries (meal/nap/diaper/incident/note), tied to a person + timestamp + author
 - documents — uploaded files (DH 680 forms, etc.), tied to a person or staff, with retention metadata
 - billing_accounts, invoices, transactions — tuition billing per family; separate from the tenant's own platform subscription record
 - messages + threads — routed by child/room per Section 9.2 workflow 7
 - compliance_rules — the Florida rule pack as structured data (ratios by age band, required document types, retention periods) — see Section 5
 - audit_log — append-only, records who accessed/changed what
-

4. API surface (high-level)

Grouped by the same workflows as PRD Section 9.2 — each becomes a REST resource group plus, where noted, a WebSocket channel for live updates:

1. **Enrollment** — /facilities/:id/enrollments (CRUD), paperwork/e-signature status, room assignment
 2. **Attendance** — /facilities/:id/attendance (check-in/out), **WebSocket channel** broadcasting live ratio per group as events land — this is what makes the Check-in/Out screen's "Live ratio" badges real instead of static
 3. **Daily records** — /people/:id/daily-records (create/list by date)
 4. **Billing** — /facilities/:id/invoices, /tenants/:id/subscription (kept separate per the PRD's flat-per-center model)
 5. **Compliance** — /facilities/:id/compliance (aggregated dashboard data), /compliance-rules/:state (the rule pack lookup)
 6. **Staff & scheduling** — /facilities/:id/staff, /staff/:id/certifications, schedule endpoints with **server-side ratio validation** — the block-scheduling-on-lapsed-cert logic (Section 9.2 workflow 6) belongs here, not just in the UI
 7. **Messaging** — /threads, /messages, plus a broadcast endpoint scoped to tenant or facility admins
 8. **Operator/cross-site** — /tenants/:id/dashboard (aggregate view powering the Operator Dashboard screen)
-

5. Compliance rule pack — from spec to implementation

PRD Section 10 specified the Florida rule pack's content (ratios, DH 680, Level 2 screening, indefinite retention default). For the build phase, this becomes a structured config table (compliance_rules), not hardcoded logic:

```
state: "FL"
ratios: [
  { ageband: "infant", maxAge: 18 (months), ratio: "1:4" },
  { ageband: "toddler", maxAge: 36 (months), ratio: "1:6" },
  { ageband: "preschool", maxAge: 5 (years), ratio: "1:15" },
  { ageband: "school_age", ratio: "1:20" }
```

```

]
requiredDocuments: [
  { type: "immunization", form: "DH 680", deadlineDays: 30, exclusionOnMiss: true }
]
staffScreening: { level: 2, rescreenIntervalYears: 5 }
retentionDefault: "indefinite"

```

Adding a second state later means inserting a new row against this same schema — the ratio-calculation service, the compliance dashboard, and the incident workflow all read from this table rather than embedding Florida-specific logic. This is the concrete implementation of PRD Section 10’s “data, not code” principle.

Still open (carried from PRD Section 14): Florida’s specific incident-report field requirements weren’t conclusively found in research — the `daily_records` incident sub-schema should be built with configurable required fields per state (reusing this same rule-pack pattern) so it isn’t a blocker, but the actual Florida field list needs a follow-up research pass before that specific screen’s validation is finalized.

6. Phased engineering roadmap

1. **Foundation** — repository set up from the Figma Make export (Section 1); Postgres schema (Section 3) migrated; auth + role-based access control scaffolded against the Section 4.2/12.2 role matrix.
2. **Core CRUD** — enrollment, people/guardians, facilities/groups wired to real data; Operator and Center dashboards reading real (if sparse) data instead of the current empty/mock states.
3. **Live attendance + ratio engine** — check-in/out with the WebSocket-driven live ratio calculation (Section 4); this is the workflow most worth getting right early since it’s the accessibility/reliability differentiator from PRD Section 6.
4. **Offline sync layer** — added once core attendance/daily-records flows are stable; retrofitting this later is explicitly the thing Section 12 warned against, so it should land before, not after, a pilot center starts relying on the product on real wifi.
5. **Billing (mocked payments)** — tuition schedule, invoice generation, the platform-subscription/family-tuition separation already correctly modeled in the Figma Make screens — built against real data before Stripe is wired in.
6. **Compliance dashboard + staff certification tracking** — reading from the `compliance_rules` table (Section 5); scheduling blocked on lapsed ratio-critical certifications, matching what the prototype already demonstrates visually.
7. **Messaging** — routed threads, broadcast capability.
8. **Real integrations** — Stripe payments, and an evaluation of Level 2 background-screening vendor APIs (Florida DCF doesn’t expose this directly — a third-party screening provider integration would be needed) — deliberately sequenced last, after the core product works end-to-end on mocked data.
9. **Pilot readiness** — the point at which PRD Section 3’s “validated prototype/design” milestone graduates to the “pilot-ready with 1–3 real centers” option that was explicitly deferred when scoping the current PRD.

7. Security, privacy, and audit requirements

- **Audit logging** (Section 3’s `audit_log` table) must be append-only and cover every read/write to a people (child) record, not just writes — this is what makes the platform credible to an operator worried about staff overreach.

- **Encryption:** at rest (database-level) and in transit (TLS everywhere) from the first deployment, not added before a specific tenant demands it.
- **Child data handling:** this product collects and stores minors' personal information (names, DOB, guardian contact info, health/immunization data, daily activity records including photos). While full COPPA applicability depends on specifics (COPPA primarily governs online services directed at children collecting data *from* children directly, which is different from a B2B tool operators and guardians use on children's behalf), the data sensitivity is real regardless of which specific statute applies — treat child PII with the same rigor as health data (access logging, minimum-necessary field exposure per role, no third-party data sharing) rather than relying on a narrow legal-applicability argument.
- **Role enforcement must be server-side.** The Section 4.2/12.2 role matrix is currently a design/UI concept in the prototype; in the build phase it must be enforced at the API layer (a Teacher's token literally cannot query another room's data), not just hidden in the frontend.

8. Decision log (this round)

Decision	Answer
Next-phase scope	Build-phase PRD first, code generation to follow in a later session
Frontend foundation	Keep Figma Make's Vite/React/TypeScript/Tailwind output — do not restart on React Native
Backend stack	Node.js + TypeScript, REST + WebSockets
Database	PostgreSQL
Payments	Stripe, deliberately sequenced late in the roadmap (Section 6, step 8)
Getting code out of Figma Make	Your action item (Section 1) — connector can't pull it directly; requires Figma Make's own export/GitHub sync

9. Open items

- **Figma Make export:** needs to actually happen (Section 1) before any code-level build-phase work can start in a future session.
- **Background-screening vendor:** no specific Level 2 screening API provider has been chosen or researched — needed before Section 6 step 8.
- **Florida incident-report field schema:** still unresearched (carried from the PRD) — needed before finalizing the Daily Logs incident sub-form validation.
- **Hosting provider (AWS vs. GCP):** recommended generically in Section 2; not yet decided specifically.
- **Timeline/resourcing:** this roadmap sequences work but doesn't estimate effort or assign it to people — worth doing once you know who's building this.